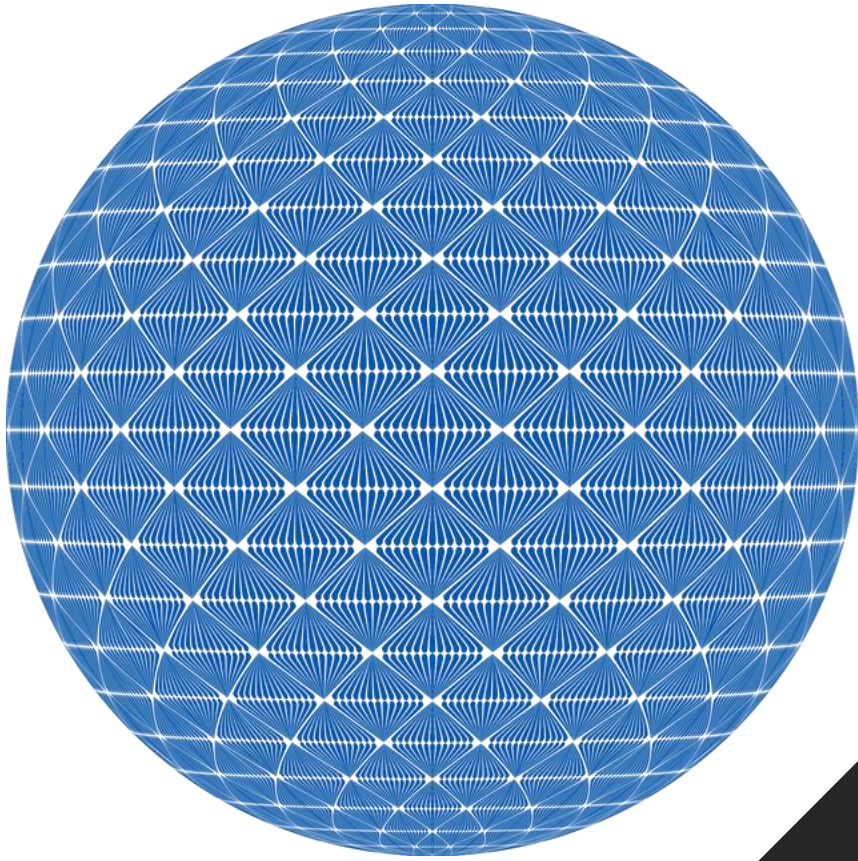


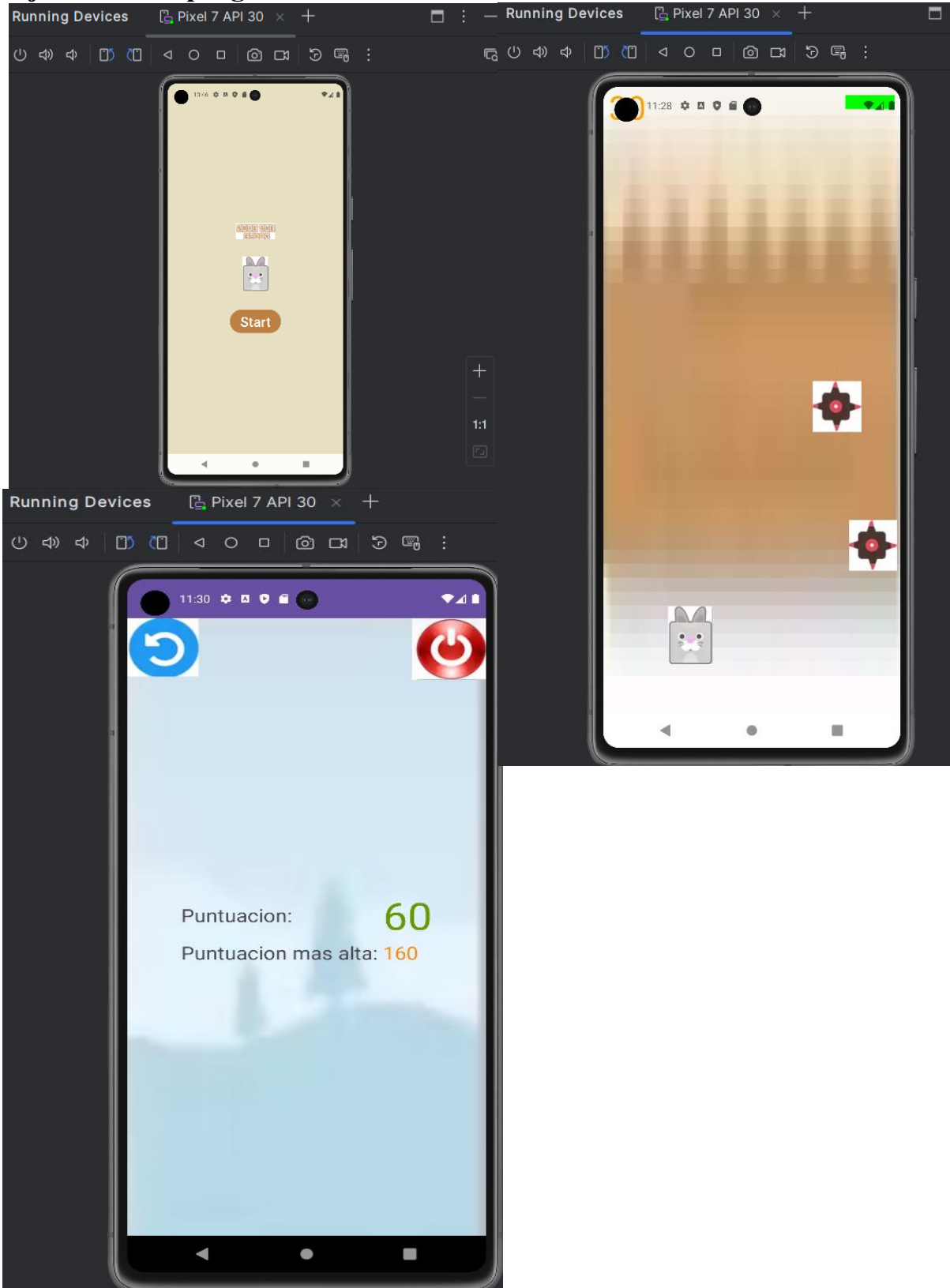


Aplicaciones Móviles

Índice

Ejecución del programa.....	2
Explicación del código.....	3
Clase Explosion.....	3
Método Explosion(Context context):	3
Método getExplosion(int explosionFrame):	3
Clase Spike	4
Spike(Context context):	4
getSpike(int spikeFrame):	4
resetPosition():	4
Clase MainActivity	5
onCreate(savedInstanceState: Bundle?):	5
startGame(view: View):	5
Clase GameOver.....	6
onCreate(@Nullable Bundle savedInstanceState):.....	6

Ejecución del programa



Explicación del código

Clase Explosion

```
1 package com.example.conejojorge;
2
3 import android.content.Context;
4 import android.graphics.Bitmap;
5 import android.graphics.BitmapFactory;
6
7 public class Explosion {
8     Bitmap[] explosion = new Bitmap[3];
9     int explosionFrame = 0;
10    int explosionX, explosionY;
11
12    @ public Explosion(Context context){
13        explosion[0] = BitmapFactory.decodeResource(context.getResources(), R.drawable.explode0);
14        explosion[1] = BitmapFactory.decodeResource(context.getResources(), R.drawable.explode1);
15        explosion[2] = BitmapFactory.decodeResource(context.getResources(), R.drawable.explode2);
16    }
17
18    public Bitmap getExplosion(int explosionFrame) {
19        if (explosionFrame < 0 || explosionFrame >= explosion.length) {
20            explosionFrame = explosion.length - 1; // Establece el último cuadro en caso de índice fuera de rango
21        }
22        return explosion[explosionFrame];
23    }
24
25
26 }
```

Esta clase gestiona la animación de una explosión en el juego. Carga tres imágenes que representan los diferentes estados de la explosión y tiene un método para obtener la imagen correspondiente según el "cuadro" de la animación que debe mostrarse.

Método Explosion(Context context):

Este es el constructor de la clase. Se ejecuta cuando creas una nueva explosión.

Dentro de este método, se cargan las tres imágenes de la explosión desde los recursos del juego y se guardan en el array explosion.

Método getExplosion(int explosionFrame):

Este método devuelve la imagen correcta de la explosión según el valor de explosionFrame.

Control de índice: Si el valor de explosionFrame es mayor que el número de imágenes disponibles (3 en total), el método ajusta el índice para que no se salga de los límites del array.

Clase Spike

```

9  public class Spike {
    7 usages
10      Bitmap spike[] = new Bitmap[4];
    4 usages
11      int spikeFrame = 0;
    5 usages
12      int spikeX, spikeY, spikeVelocity;
    4 usages
13      Random random;
    1 usage
14  @ public Spike(Context context){
15      spike [0] = BitmapFactory.decodeResource(context.getResources(), R.drawable.spike0);
16      spike [1] = BitmapFactory.decodeResource(context.getResources(), R.drawable.spike1);
17      spike [2] = BitmapFactory.decodeResource(context.getResources(), R.drawable.spike2);
18      spike [3] = BitmapFactory.decodeResource(context.getResources(), R.drawable.spike1);
19      random = new Random();
20      resetPosition();
21  }
    1 usage
22  > public Bitmap getSpike(int spikeFrame) { return spike[spikeFrame]; }
    4 usages
25  > public int getSpikeWidth() { return spike[0].getWidth(); }
    3 usages
28  public void resetPosition(){
29      spikeX = random.nextInt( bound: GameView.dWidth - getSpikeWidth());
30      spikeY = -200 + random.nextInt( bound: 600)* -1;
31      spikeVelocity = 5 *random.nextInt( bound: 16);
32  }
33
34
    1 usage
35  > public int getSpikeHeight() { return spike[0].getHeight(); }

```

La clase Spike maneja las imágenes y posiciones de los picos que caen en el juego. Los métodos más importantes son `resetPosition()` para mover los picos de forma aleatoria y `getSpike()` para obtener las imágenes de los picos que se mostrarán en pantalla.

`Spike(Context context):`

Este es el constructor de la clase. Se encarga de cargar las imágenes de los picos desde los recursos y las coloca en el arreglo `spike[]`.

Además, se crea el objeto `random` para generar valores aleatorios.

Finalmente, llama a `resetPosition()` para posicionar los picos en una posición inicial aleatoria.

`getSpike(int spikeFrame):`

Este método devuelve la imagen del pico que corresponde al índice `spikeFrame`.

Esto se usa para mostrar la animación de los picos cambiando entre las imágenes disponibles.

`resetPosition():`

Este método posiciona los picos en una posición aleatoria en la parte superior de la pantalla.

`spikeX` obtiene un valor aleatorio entre 0 y el ancho de la pantalla (`GameView.dWidth`).

`spikeY` obtiene una posición aleatoria hacia arriba en la pantalla.

`spikeVelocity` es una velocidad aleatoria para los picos que hace que caigan a diferentes velocidades.

Clase MainActivity

```
1 package com.example.conejojorge
2
3 > import ...
10
11 class MainActivity : AppCompatActivity() {
12     override fun onCreate(savedInstanceState: Bundle?) {
13         super.onCreate(savedInstanceState)
14         enableEdgeToEdge()
15         setContentView(R.layout.activity_main)
16         getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
17     }
18
19     fun startGame(view: View) {
20         val gameView = GameView(context: this)
21         setContentView(gameView)
22     }
23 }
24 }
```

MainActivity es la actividad inicial donde se muestra la pantalla de inicio.

El botón de "Start" cambia la vista a la pantalla del juego (GameView) cuando se presiona.

onCreate(savedInstanceState: Bundle?):

Este es el método que se ejecuta cuando la actividad (pantalla) se crea.

Llama al método `super.onCreate(savedInstanceState)` para asegurar que la actividad se cree correctamente.

`enableEdgeToEdge()`: Activa el modo de pantalla completa, eliminando las barras de navegación y estado (puede ser para dar una mejor experiencia visual).

`setContentView(R.layout.activity_main)`: Establece el layout XML (`activity_main.xml`) como la interfaz de usuario inicial, es decir, la pantalla de inicio con los elementos (como el botón "Start").

`getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)`: Impide que la pantalla se apague mientras el juego o la actividad principal esté activa.

startGame(view: View):

Este método se llama cuando el usuario presiona el botón de "Start" en el layout.

Crea una nueva instancia de la clase `GameView`, que es la vista donde se juega el juego.

`setContentView(gameView)`: Cambia la vista actual a `GameView`, lo que inicia el juego en lugar de la pantalla de inicio

Clase GameOver

```

14 public class GameOver extends AppCompatActivity {
15
16     2 usages
17     TextView tvPoints;
18     2 usages
19     TextView tvHighest;
20     3 usages
21     SharedPreferences sharedPreferences;
22     2 usages
23     ImageView ivNewHighest;
24     public void onCreate(@Nullable Bundle savedInstanceState){
25         super.onCreate(savedInstanceState);
26         setContentView(R.layout.game_over);
27         tvPoints = findViewById(R.id.tvPoints);
28         tvHighest = findViewById(R.id.tvHighest);
29         ivNewHighest = findViewById(R.id.ivNewHighest);
30         int points = getIntent().getExtras().getInt( key: "points");
31         tvPoints.setText(""+points);
32         sharedPreferences = getSharedPreferences( name: "my_pref", mode: 0);
33         int highest = sharedPreferences.getInt( key: "highest", defValue: 0);
34         if (points > highest){
35             ivNewHighest.setVisibility(View.VISIBLE);
36             highest = points;
37             SharedPreferences.Editor editor = sharedPreferences.edit();
38             editor.putInt("highest",highest);
39             editor.commit();
40         }
41         tvHighest.setText(""+highest);
42     }
43     1 usage
44     public void restart(View view){
45         Intent intent = new Intent( packageContext: GameOver.this,MainActivity.class);

```

Esta clase maneja la pantalla de "Game Over", que se muestra cuando el jugador pierde el juego. Aquí se muestran los puntos obtenidos y se verifica si el jugador ha alcanzado un nuevo puntaje más alto.

onCreate(@Nullable Bundle savedInstanceState):

Este es el método que se ejecuta cuando la actividad se crea, es decir, cuando la pantalla de "Game Over" es cargada.

setContentView(R.layout.game_over): Establece el layout XML correspondiente a la pantalla de "Game Over".

Inicialización de los elementos de la interfaz:

tvPoints: Muestra los puntos obtenidos en el juego.

tvHighest: Muestra el puntaje más alto almacenado en las preferencias compartidas.

ivNewHighest: Una imagen que indica si se ha alcanzado un nuevo puntaje más alto.

Obtención de los puntos:

Los puntos obtenidos en la partida actual se pasan desde la actividad anterior a través de un Intent, y se muestran en tvPoints.

Uso de SharedPreferences:

Se obtiene el puntaje más alto guardado previamente con sharedPreferences.getInt("highest", 0).

Si el puntaje actual es mayor que el puntaje más alto, se actualiza el puntaje más alto en las preferencias y se muestra una imagen (ivNewHighest) indicando que se ha alcanzado un nuevo puntaje.

Guardar el nuevo puntaje más alto:

Si se logra un nuevo puntaje más alto, se actualiza la preferencia guardada y se guarda el valor con `editor.putInt("highest", highest)`.

`restart(View view):`

Este método se llama cuando el jugador quiere reiniciar el juego.

Crea un Intent para volver a la actividad principal (`MainActivity`), lo que reinicia el juego.

`startActivity(intent)` lanza la actividad `MainActivity`, y `finish()` cierra la actividad actual de "Game Over".

`exit(View view):`

Este método se llama cuando el jugador quiere salir del juego.

Simplemente cierra la actividad actual con `finish()`, lo que finaliza la aplicación.

Clase GameView

```

23 public class GameView extends View {
44
45     public GameView(Context context) {
46         super(context);
47         this.context = context;
48         background = BitmapFactory.decodeResource(getResources(), R.drawable.background);
49         ground = BitmapFactory.decodeResource(getResources(), R.drawable.ground);
50         rabbit = BitmapFactory.decodeResource(getResources(), R.drawable.rabbit);
51         Display display = ((Activity) getContext()).getWindowManager().getDefaultDisplay();
52         Point size = new Point();
53         display.getSize(size);
54         dWidth = size.x;
55         dHeight = size.y;
56         rectBackground = new Rect(0, 0, dWidth, dHeight);
57         rectGround = new Rect(0, dHeight - ground.getHeight(), dWidth, dHeight);
58         handler = new Handler();
59         runnable = new Runnable() {
60             @Override
61             public void run() { invalidate(); }
62         };
63
64         textPaint.setColor(Color.rgb(255, 165, 0));
65         textPaint.setTextSize(TEXT_SIZE);
66         textPaint.setTextAlign(Paint.Align.LEFT);
67         //textPaint.setTypeface(ResourcesCompat.getFont(context, R.font.kenney_blocks));
68         healthPaint.setColor(Color.GREEN);
69         random = new Random();
70         rabbitX = dWidth / 2 - rabbit.getWidth() / 2;
71         rabbitY = dHeight - ground.getHeight() - rabbit.getHeight();
72         spikes = new ArrayList<>();
73         explosions = new ArrayList<>();
74         for (int i = 0; i < 3; i++) {
75             Spike spike = new Spike(context);
76
23 public class GameView extends View {
45     public GameView(Context context) {
74         explosions = new ArrayList<>();
75         for (int i = 0; i < 3; i++) {
76             Spike spike = new Spike(context);
77             spikes.add(spike);
78         }
79     }
80
81     @Override
82     protected void onDraw(Canvas canvas){
83         super.onDraw(canvas);
84         canvas.drawBitmap(background, null, rectBackground, null);
85         canvas.drawBitmap(ground, null, rectBackground, null);
86         canvas.drawBitmap(rabbit, rabbitX, rabbitY, null);
87         for(int i = 0; i < spikes.size(); i++){
88             canvas.drawBitmap(spikes.get(i).getSpike(spikes.get(i).spikeFrame), spikes.get(i).spikeX, spikes.get(i).spikeY, null);
89             spikes.get(i).spikeFrame++;
90             if (spikes.get(i).spikeFrame > 2){
91                 spikes.get(i).spikeFrame = 0;
92             }
93             spikes.get(i).spikeY += spikes.get(i).spikeVelocity;
94             if (spikes.get(i).spikeY + spikes.get(i).getSpikeHeight() >= dHeight - ground.getHeight()){
95                 points += 10;
96                 Explosion explosion = new Explosion(context);
97                 explosion.explosionX = spikes.get(i).spikeX;
98                 explosion.explosionY = spikes.get(i).spikeY;
99                 explosions.add(explosion);
100                 spikes.get(i).resetPosition();
101             }
102         }
103     }

```

sample > conejojorge > GameView > spikes 70:31 CRLF UTF-8 4 spaces

Este código define la vista del juego donde se manejan todos los elementos visuales y la lógica del juego, como el movimiento del conejo, la caída de las espinas, y la puntuación del jugador. Además, gestiona la interacción táctil del jugador con la pantalla.

GameView(Context context):

Constructor de la clase que inicializa las imágenes, las dimensiones de la pantalla y la lógica de las espinas y explosiones.

Llama al método resetPosition para inicializar las espinas con posiciones aleatorias.

onDraw(Canvas canvas):

Este es el método principal para actualizar la pantalla.

Dibuja el fondo, el suelo y el conejo.

Gestión de espinas:

Cada espina se mueve hacia abajo en la pantalla. Si llega al suelo, se genera una explosión y la espina se reinicia a una nueva posición aleatoria.

Se detectan colisiones entre las espinas y el conejo. Si ocurre una colisión, se reduce la vida del jugador y se reinicia la espina.

Explosiones:

Se muestran las explosiones cuando las espinas tocan el suelo, y se eliminan después de unos fotogramas.

Actualización de la vida y puntuación:

La vida restante del jugador se dibuja en un rectángulo. Además, se dibuja la puntuación en la pantalla.

Actualización periódica:

Después de cada dibujo, se programa una actualización para que el juego continúe con un retraso definido (UPDATE_MILIS).

onTouchEvent(MotionEvent event):

Este método maneja los eventos táctiles. El jugador puede mover al conejo tocando y arrastrando en la pantalla.

Si el dedo se mueve de izquierda a derecha, el conejo sigue ese movimiento, pero se limita para que no se salga de la pantalla (usando condiciones para la posición rabbitX).

Utiliza variables como oldX y oldRabbitX para calcular el movimiento relativo.